TaskForge Quest System

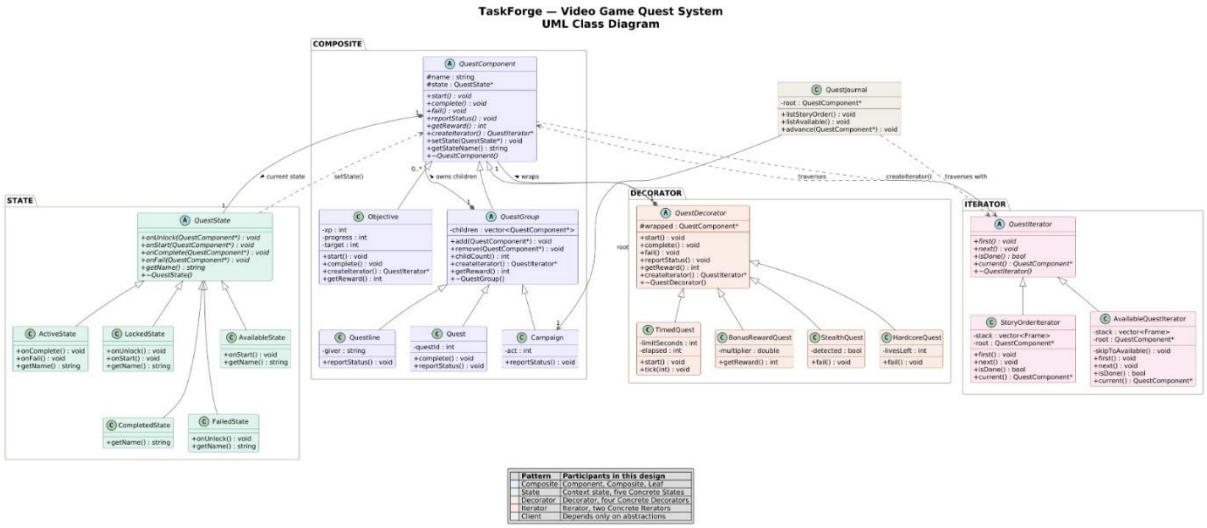
UML & Workflow Documentation

Design Patterns, Runtime Structure, State Lifecycle, and Activity Workflows

Four GoF Patterns Integrated:

Composite • State • Decorator • Iterator

Diagram 1: UML Class Diagram



Overview

The design integrates four GoF patterns around a single QuestComponent abstraction:

Composite Pattern

The hierarchy is rooted at QuestComponent (Component interface). QuestGroup is the Composite that owns vector<QuestComponent*> children. The concrete Composites are Campaign (root), Questline (middle), and Quest (lowest). Objective is the Leaf with no children. All recursive operations (start(), complete(), fail(), getReward(), isSatisfied(), countQuests()) are delegated from parent to children.

State Pattern

Each QuestComponent holds a QuestState* state pointer (making it the Context). QuestState is the abstract State interface. Concrete states are LockedState, AvailableState, ActiveState, CompleteState, and FailedState. State transitions happen when a lifecycle method (e.g., onStart(), onComplete()) calls quest->setState(newState) on the context.

Decorator Pattern

QuestDecorator extends QuestComponent and wraps another QuestComponent* wrapped (composition, not inheritance). Each decorator delegates all operations to its wrapped component and adds one specific behavior. Concrete decorators are TimedQuest (enforces time limits), BonusRewardQuest (multiplies rewards),

StealthQuest (conditional failure), and HardcoreQuest (absorbs failures). Multiple decorators can be stacked on a single quest at runtime.

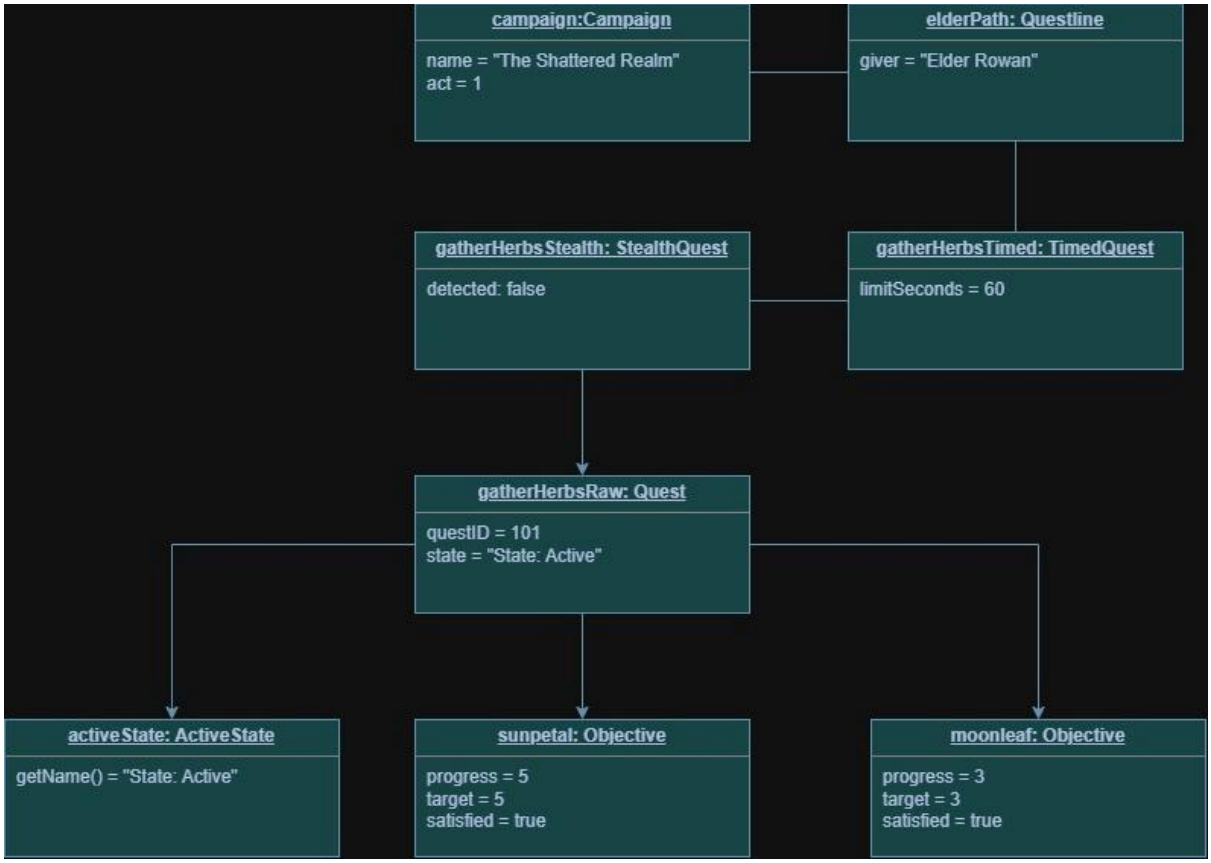
Iterator Pattern

QuestIterator is the abstract iterator with methods first(), next(), isDone(), and current(). StoryOrderIterator does recursive depth-first traversal using nested child iterators. SingleIterator handles leaves (Objectives). AvailableQuestIterator filters traversal to only State: Available components. Iterators encapsulate traversal logic; client code never calls getChild() directly.

Key Integration

A single pointer type (QuestComponent*) allows the client (QuestJournal) to work with Campaigns, Questlines, Quests, Objectives, and any decorated variant identically—client code cannot tell them apart and never needs to.

Diagram 2: UML Object Diagram (Runtime Snapshot)



Overview

This object diagram captures a meaningful runtime state partway through gameplay:

The Snapshot Structure

campaign: Campaign ("The Shattered Realm", act=1)
  -> elderPath: Questline (giver="Elder Rowan")
    -> gatherHerbsTimed: TimedQuest (limitSeconds=60)
      -> gatherHerbsStealth: StealthQuest (detected=false)
        -> gatherHerbsRaw: Quest (questId=101)
          -> activeState: ActiveState (getName() = “State: Active”)

-> sunpetal: Objective (progress=5/5, satisfied)
-> moonleaf: Objective (progress=3/3, satisfied)

Pattern Visibility

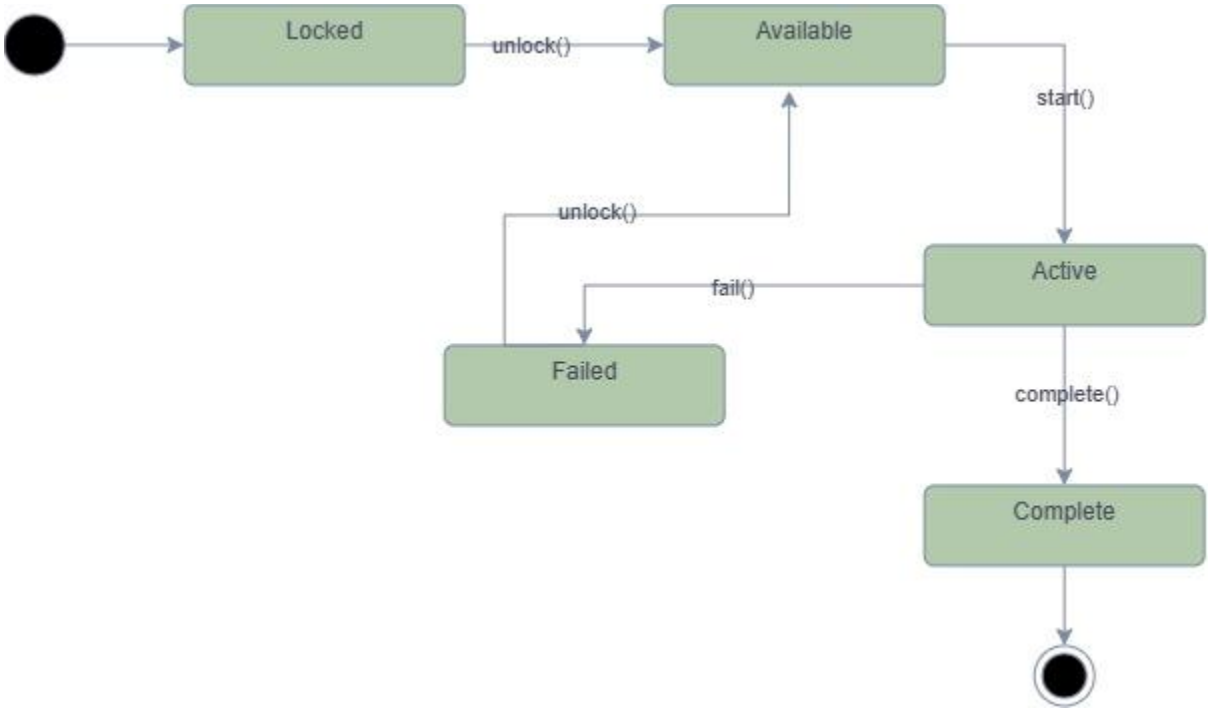
Composite: The tree structure shows parent-child relationships (aggregation). Each QuestComponent holds zero or more children of type QuestComponent*. Objectives are leaves; Quests and Questlines are composites.

State: Each Quest object holds a reference to its current state object. For example, gatherHerbsRaw.state points to an ActiveState instance. This enables polymorphic behavior: calling gatherHerbsRaw->complete() delegates to the state's onComplete(), which transitions the quest based on its current mode.

Decorator: The decorator chain is visible as nested wrapped pointers. gatherHerbsTimed wraps gatherHerbsStealth, which wraps gatherHerbsRaw. Each decorator delegates to the next layer. When getReward() is called on gatherHerbsTimed, it eventually queries gatherHerbsRaw, but each layer can modify the result.

Iterator: Not visible in this static snapshot, but iterators would walk this tree structure. A StoryOrderIterator starting at campaign would visit nodes in depth-first order; an AvailableQuestIterator would skip any quest not in State: Available.

Diagram 3: UML State Diagram (Quest Lifecycle)



Overview

This state machine diagram models the lifecycle of a Quest object (the Context in the State pattern):

States and Their Meaning

Locked: The quest exists but the player has not yet earned the right to accept it. Calling start(), complete(), or fail() has no effect. Only unlock() transitions to Available.

Available: The quest is unlocked and the player can accept it. Calling start() transitions to Active. All other operations have no effect.

Active: The quest is running. The player is making progress on its objectives. Calling complete() transitions to Complete. Calling fail() transitions to Failed.

Complete: Terminal state. The quest succeeded. All operations have no effect.

Failed: Terminal state. The quest failed. Calling unlock() transitions back to Available (giving the player a retry).

Transitions and Events

quest->unlock(): Locked → Available (or no-op in other states)

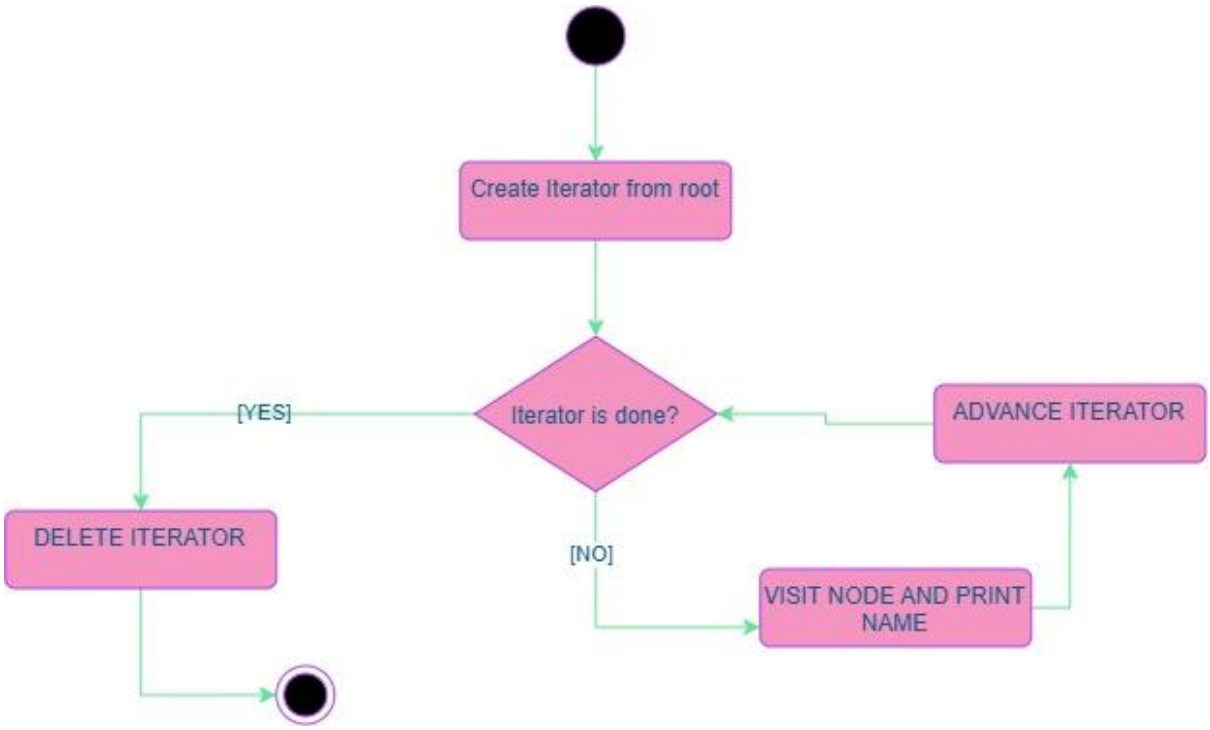
quest->start(): Available → Active (or no-op in other states)

quest->complete(): Active → Complete (or no-op in other states)

quest->fail(): Active → Failed (or no-op in other states)

Implementation Detail: Invalid transitions are handled gracefully by implementing the do nothing behavior in each state. For example, LockedState::onComplete() is empty—it does not throw an exception. This is the safety variant: the state object knows what it should do in response to each event.

Diagram 4: Activity Diagram (Iterator Traversal Workflow)



Overview

This activity diagram makes the traversal mechanism visible as part of a larger workflow, demonstrating how the Iterator pattern encapsulates tree navigation:

The Workflow

1. Initialize: The client (e.g., QuestJournal::listStoryOrder()) calls root->createIterator(), which returns a new StoryOrderIterator pointing at the root.

2. Loop Entry: The workflow enters a loop by calling iterator->first(), which initializes the iterator to point at the first node in traversal order.

3. Loop Body (repeats):

Decision: Check iterator->isDone(). If true, exit loop. If false, continue.

Process: Call iterator->current() to retrieve the current QuestComponent* node and process it.

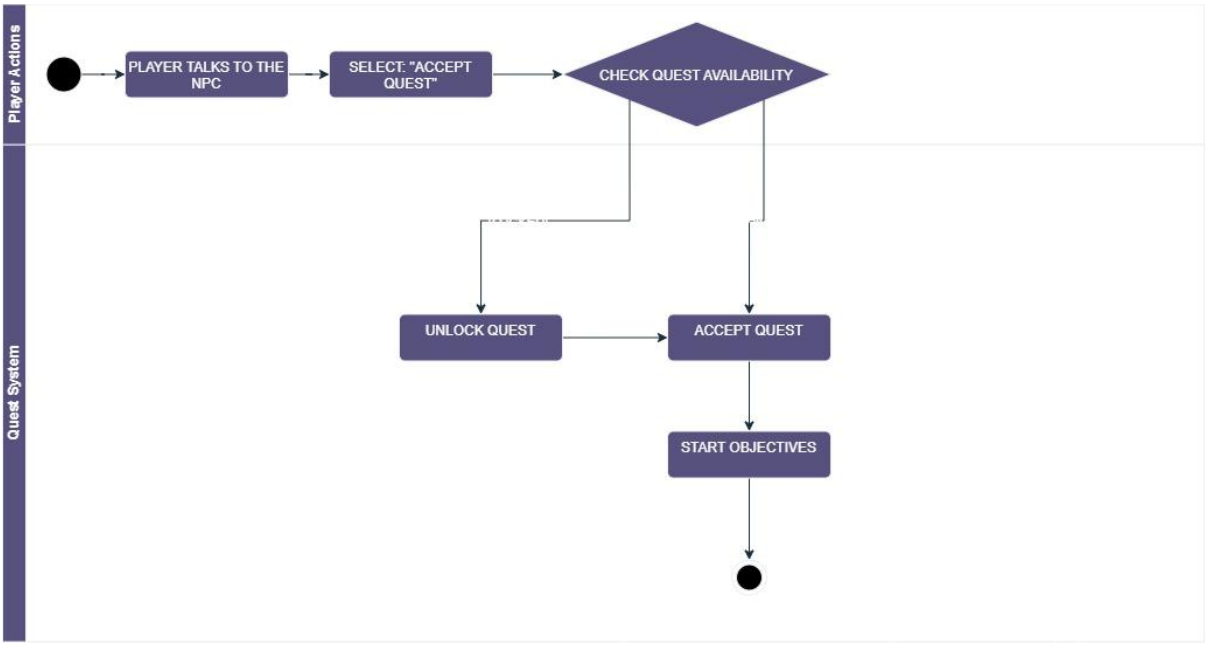
Advance: Call iterator->next() to move to the next node in traversal order.

4. Loop Exit: When isDone() returns true, the workflow ends and the iterator is deleted.

Key Property: The client code never calls getChild(), never manages a vector<>, and never recurses explicitly. The iterator hides the tree structure entirely.

Alternative Traversal: AvailableQuestIterator uses the same loop interface but filters nodes. It wraps a StoryOrderIterator and skips any node whose getStateName() is not State: Available.

Diagram 5: Activity Diagram (Quest Acceptance Workflow)



Overview

This activity diagram models the workflow when a player accepts a quest, using swimlanes to show the two principal actors (Player and Quest System):

Player Swimlane (initiates the action)

1. Player talks to an NPC.
2. Player selects Accept Quest from the dialogue.
3. The action passes to the Quest System swimlane.

Quest System Swimlane (processes the action)

1. Receive the Accept Quest request.
2. Decision: Is the quest in Locked state or Available state?

   If Locked: Execute Unlock Quest → transitions quest from Locked to Available.

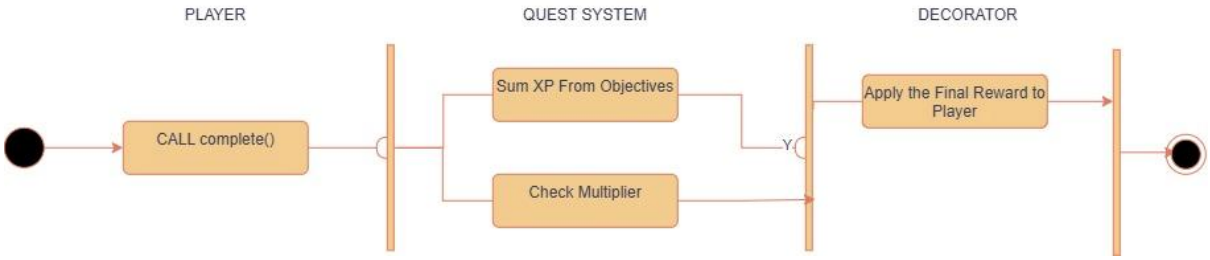
   If Available: Skip unlock (quest is already available).
3. Both paths merge at Accept Quest.

4. Execute Start Quest → call quest->start(), which transitions to Active and cascades to child objectives.

Workflow Semantics

A locked quest cannot be started directly; the player must unlock it first. Once available, starting a quest immediately transitions it to active and cascades the start action through all descendants.

Diagram 6: Activity Diagram (Completion & Reward Calculation)



Overview

This activity diagram models the quest completion workflow, showing how the Composite and Decorator patterns collaborate in calculating the final reward:

The Workflow

1. Initiate: Player calls quest->complete() (or the game triggers it).

2. State Transition: The quest's current state is asked to handle completion. For a quest in Active state, ActiveState::onComplete() transitions the quest to Complete state.

3. Conditional Execution: The state transition is verified. Only if the state actually changed AND the quest is now in Complete state does the following execute:

Path 1 (Composite Recursion): Call QuestGroup::complete(), which recursively calls complete() on every child. Each Objective marks itself complete and triggers reward calculation.

Path 2 (Decorator Evaluation): Call getReward() on the quest. For a decorated quest (e.g., BonusRewardQuest), the decorator multiplies the base reward by its multiplier.

4. Synchronize: Both paths conclude. The quest's total XP is now known and is awarded to the player.

Composite Contribution

The tree structure allows recursive completion: calling complete() on a Quest automatically completes all child Objectives. Each Objective returns its XP value only if satisfied. The Quest sums these values via its overridden getReward() method.

Decorator Contribution

A decorated quest (e.g., BonusRewardQuest(quest, 1.5)) intercepts the getReward() call and multiplies by its modifier before returning. If the quest is wrapped in multiple decorators, each layer can apply its own logic.